INDEX

Sr. No	Artificial Intelligence Section	Date	CO	Sign
1.	Implementation of Logic programming using PROLOG DFS for water jug problem		CO1	
2.	Implementation of Logic programming using PROLOG BFS for tic-tac-toe problem		CO1	
3.	Implementation of Logic programming using PROLOG Hill-climbing to solve 8- Puzzle Problem.		CO1	
4.	Introduction to Python Programming: Learn the different libraries - NumPy, Pandas, SciPy, Matplotlib, Scikit Learn.		CO2	
5.	Implement Perceptron algorithm for OR operation		CO2	
6.	Improve the prediction accuracy by estimating the weight values for the training data using stochastic gradient descent. (Perceptron)		CO2	
7.	Implement Adaline algorithm for AND operation		CO2	

Sr. No	Machine Learning Section	Date	CO	Sign
1.	Implementation of Features Extraction and Selection, Normalization, Transformation, Principal Components Analysis.		CO3	
2.	Implementation of Logistic regression		CO4	
3.	Implementation of Classifying data using Support Vector Machine (SVM)- Linear and Non-Linear SVM Classification		CO4	
4.	Implement Elbow method for K means Clustering		CO4	
5.	Implementation of Bagging Algorithm: Random Forest		CO4	
6.	Implementation of Boosting Algorithms: AdaBoost, Stochastic Gradient Boosting, Voting Ensemble		CO4	

Artificial Intelligence Section

PRACTICAL NO 1

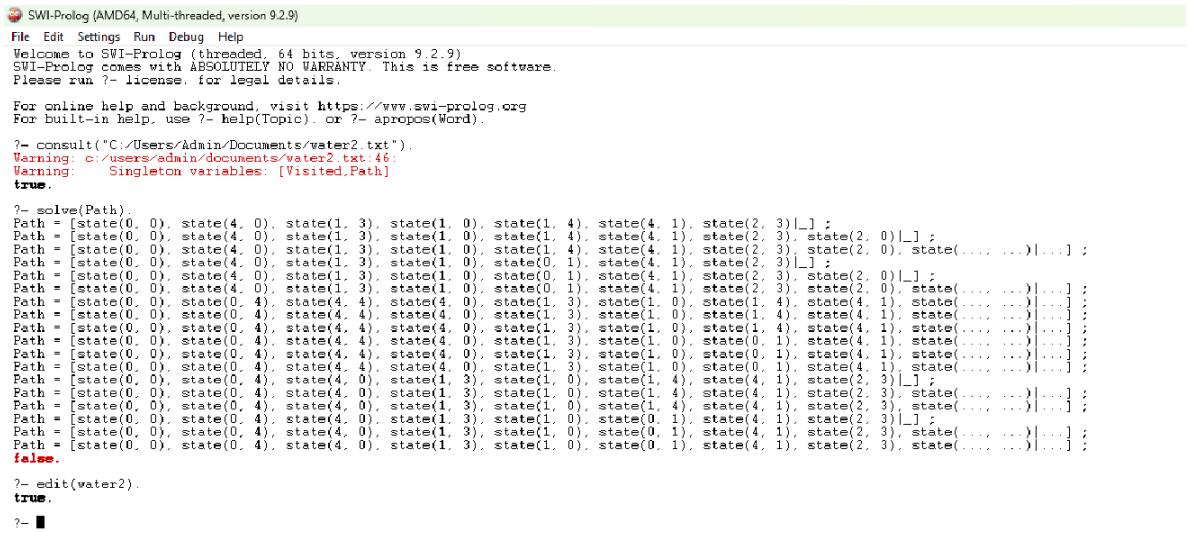
Implementation of Logic programming using PROLOG DFS for water jug problem

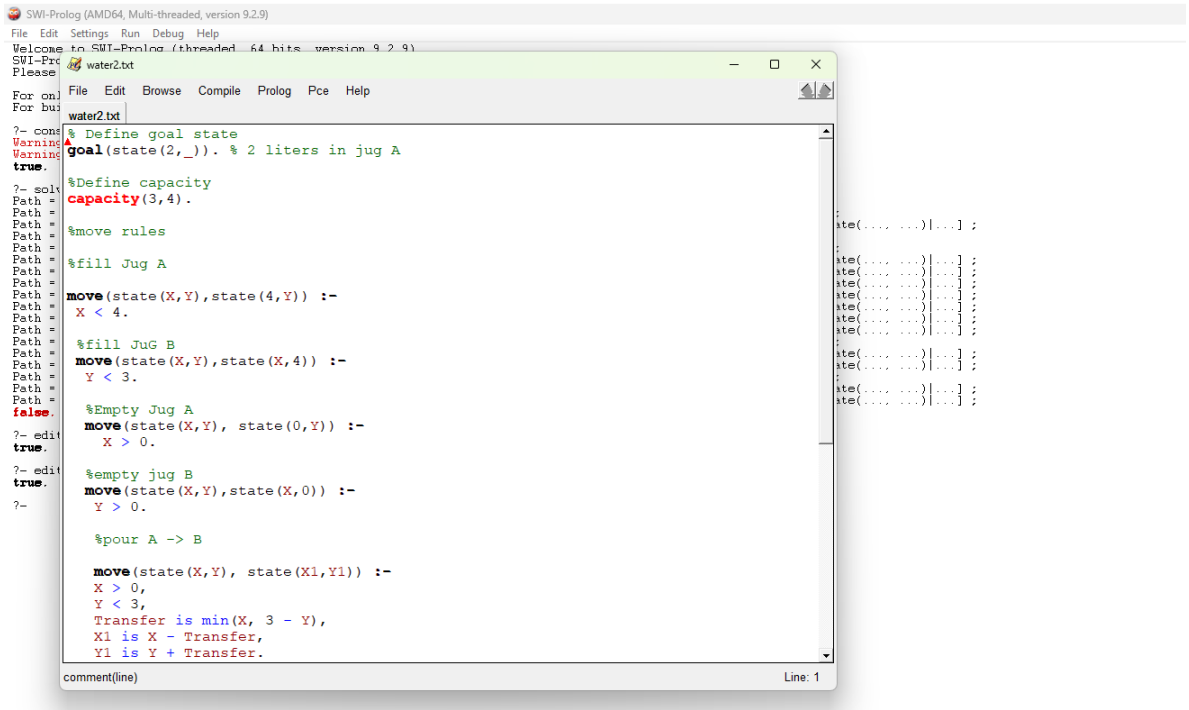
CODE:

```
/* Water Jug Problem using Prolog */

/* Goal state: 2 liters in Jug A */
goal(state(2,_)).

/* Capacities of jugs */
capacity(4,3).

/* Move rules */

/* Fill Jug A */
move(state(X,Y), state(4,Y)) :-
   X < 4.

/* Fill Jug B */
move(state(X,Y), state(X,3)) :-
   Y < 3.

/* Empty Jug A */
move(state(X,Y), state(0,Y)) :-
   X > 0.

/* Empty Jug B */
move(state(X,Y), state(X,0)) :-
   Y > 0.

/* Pour A -> B */
move(state(X,Y), state(X1,Y1)) :-
   X > 0,
   Y < 3,
   Transfer is min(X, 3-Y),
   X1 is X - Transfer,
   Y1 is Y + Transfer.

/* Pour B -> A */
move(state(X,Y), state(X1,Y1)) :-
   Y > 0,
   X < 4,
   Transfer is min(Y, 4-X),
   Y1 is Y - Transfer,
   X1 is X + Transfer.

/* Path search */
path(State, _) :-
   goal(State),
   write('Goal reached: '), write(State), nl.
```

```
path(State, Visited) :-
    move(State, NextState),
    \+ member(NextState, Visited),
    write(NextState), nl,
    path(NextState, [NextState|Visited]).

/* Start */
waterjug :-
    path(state(0,0), [state(0,0)]).
```

OUTPUT:





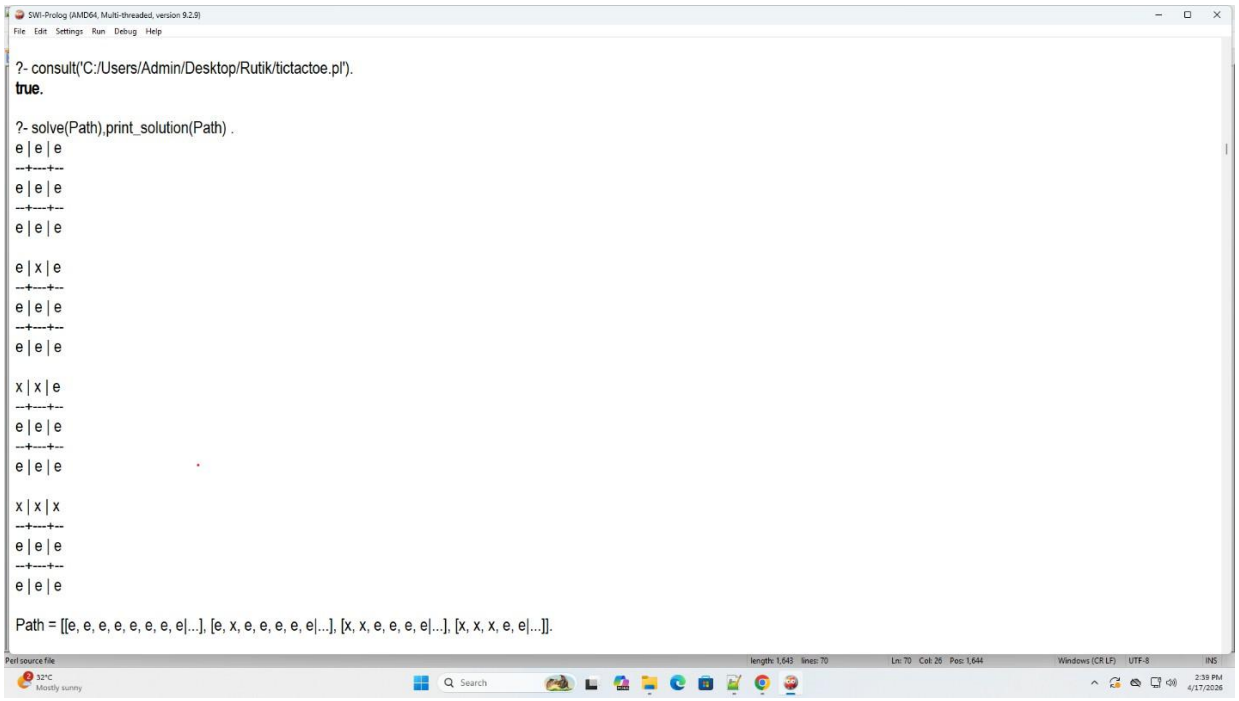
PRACTICAL NO 2
Implementation of Logic programming using PROLOG BFS for tic-tac-toe problem

CODE:

```
% --- Tic Tac Toe BFS Solver ---

% Initial empty board
initial([e,e,e,
         e,e,e,
         e,e,e]).

% Goal: X wins
goal(Board) :- win(Board, x).

% Winning conditions
win([P,P,P,_,_,_,_,_,_],P).
win([_,_,_,P,P,P,_,_,_],P).
win([_,_,_,_,_,_,P,P,P],P).
win([P,_,_,P,_,_,P,_,_],P).
win([_,P,_,_,P,_,_,P,_],P).
win([_,_,P,_,_,P,_,_,P],P).
win([P,_,_,_,P,_,_,_,P],P).
win([_,_,P,_,P,_,P,_,_],P).

% Switch player
next_player(x, o).
next_player(o, x).

% Generate next moves
move(Board, Player, NewBoard) :-
    nth0(Index, Board, e),          % find empty cell
    replace(Board, Index, Player, NewBoard).

% Replace element in list
replace([_|T], 0, X, [X|T]).
replace([H|T], I, X, [H|R]) :-
    I > 0,
    I1 is I - 1,
    replace(T, I1, X, R).

% --- BFS Implementation ---

bfs([[Board|Path]|_], _, [Board|Path]) :-
    goal(Board), !.

bfs([[Board|Path]|Queue], Player, Solution) :-
    findall(
        [NextBoard,Board|Path],
        move(Board, Player, NextBoard),
        Children
    ),
    append(Queue, Children, NewQueue),
```

```
    next_player(Player, NextPlayer),
    bfs(NewQueue, NextPlayer, Solution).

% Solve predicate
solve(Path) :-
    initial(Board),
    bfs([[Board]], x, RevPath),
    reverse(RevPath, Path).

% Print board
print_board([A,B,C,D,E,F,G,H,I]) :-
    format("~w | ~w | ~w~n",[A,B,C]),
    format("--+---+--~n"),
    format("~w | ~w | ~w~n",[D,E,F]),
    format("--+---+--~n"),
    format("~w | ~w | ~w~n~n",[G,H,I]).

% Print solution path
print_solution([]).
print_solution([Board|Rest]) :-
    print_board(Board),
    print_solution(Rest).
```

OUTPUT-



PRACTICAL NO 3

Implementation of Logic programming using PROLOG Hill-climbing to solve 8- Puzzle Problem.

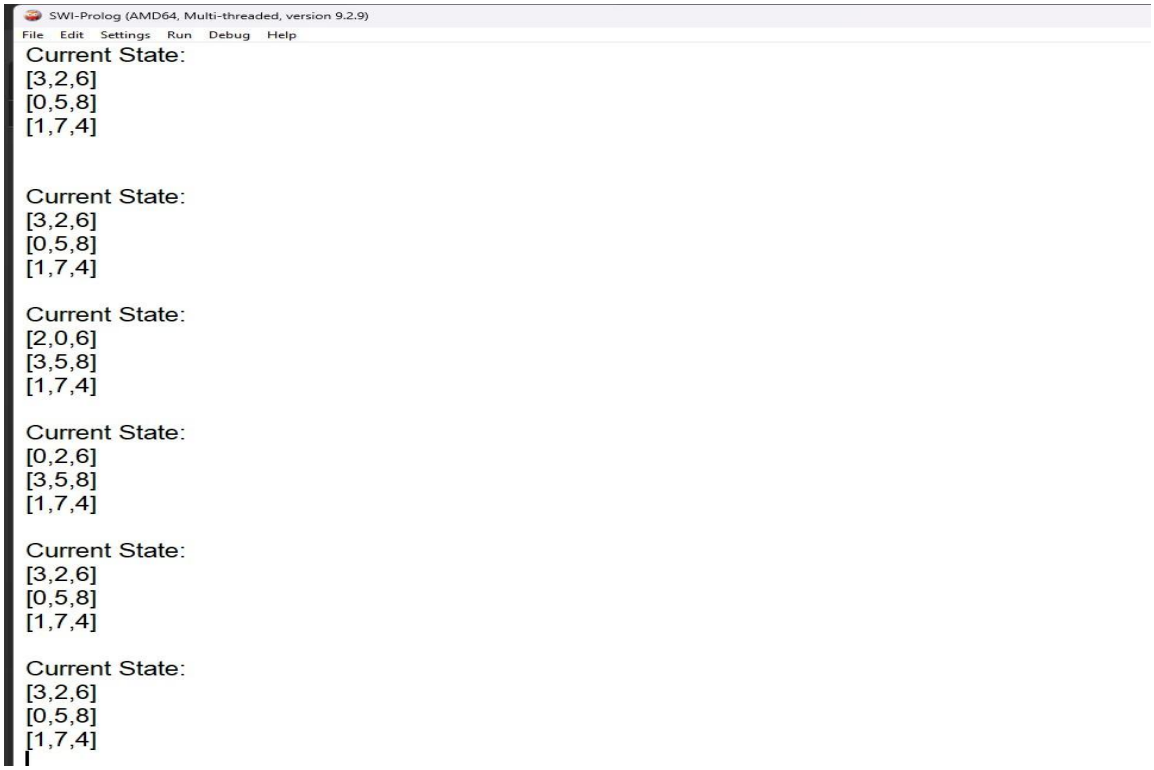
CODE:

```
% Goal State (as a flat list)
goal([1,2,3,4,5,6,7,8,0]).  % 0 represents blank

% Entry point
solve(Start) :-
    hill_climb(Start, []).

% Hill Climbing Algorithm
hill_climb(State, _) :-
    goal(State),
    write('Goal Reached:'), nl,
    print_state(State).

hill_climb(State, Visited) :-
    get_children(State, Children),
    exclude(member_of(Visited), Children, ValidChildren),
    evaluate_children(ValidChildren, ScoredChildren),
    sort(ScoredChildren, Sorted),   % sort by heuristic
    Sorted = [[_, BestChild] | _],  % pick best child
    write('Current State:'), nl,
    print_state(State), nl,
    hill_climb(BestChild, [State | Visited]).

% Check membership
member_of(List, Element) :-
    member(Element, List).

% Generate children (possible moves)
get_children(State, Children) :-
    findall(Child, move(State, Child), Children).

% Possible Moves
move(State, NewState) :-
    swap(State, 0, -3, NewState). % up
move(State, NewState) :-
    swap(State, 0, 3, NewState).  % down
move(State, NewState) :-
    swap(State, 0, -1, NewState). % left
move(State, NewState) :-
    swap(State, 0, 1, NewState).  % right

% Swap positions
swap(State, Zero, Move, NewState) :-
    nth0(Index, State, Zero),
    NewIndex is Index + Move,

    valid_move(Index, NewIndex),
```

```
    nth0(NewIndex, State, Value),
    set_element(State, Index, Value, Temp),

    set_element(Temp, NewIndex, Zero, NewState).

% Valid moves
valid_move(I, J) :-
    J >= 0, J < 9,
    \+ invalid_edge(I, J).

invalid_edge(2, 3).
invalid_edge(3, 2).
invalid_edge(5, 6).
invalid_edge(6, 5).

% Replace element in list
set_element([_|T], 0, X, [X|T]).
set_element([H|T], I, X, [H|R]) :-
    I > 0,
    I1 is I - 1,
    set_element(T, I1, X, R).

% Heuristic: misplaced tiles
heuristic(State, H) :-
    goal(Goal),
    count_misplaced(State, Goal, H).

count_misplaced([], [], 0).
count_misplaced([0|T1], [_|T2], H) :-
    count_misplaced(T1, T2, H).
count_misplaced([H|T1], [H|T2], C) :-
    count_misplaced(T1, T2, C).
count_misplaced([H1|T1], [H2|T2], C) :-
    H1 \= H2,
    count_misplaced(T1, T2, C1),
    C is C1 + 1.

% Evaluate children
evaluate_children([], []).
evaluate_children([State|Rest], [[H, State] | ScoredRest]) :-
    heuristic(State, H),
    evaluate_children(Rest, ScoredRest).

% Print state
print_state([A,B,C,D,E,F,G,H,I]) :-
    write([A,B,C]), nl,
    write([D,E,F]), nl,
    write([G,H,I]), nl.
```

OUTPUT-

```
?- consult("C:/Users/Admin/Desktop/Rutik/Hill_Climbing").
true.

?- solve([1,2,3,4,5,6,0,7,8]).
Current State:
[1,2,3]
[4,5,6]
[0,7,8]

Current State:
[1,2,3]
[4,5,6]
[7,0,8]

Goal Reached:
[1,2,3]
[4,5,6]
[7,8,0]
true
```



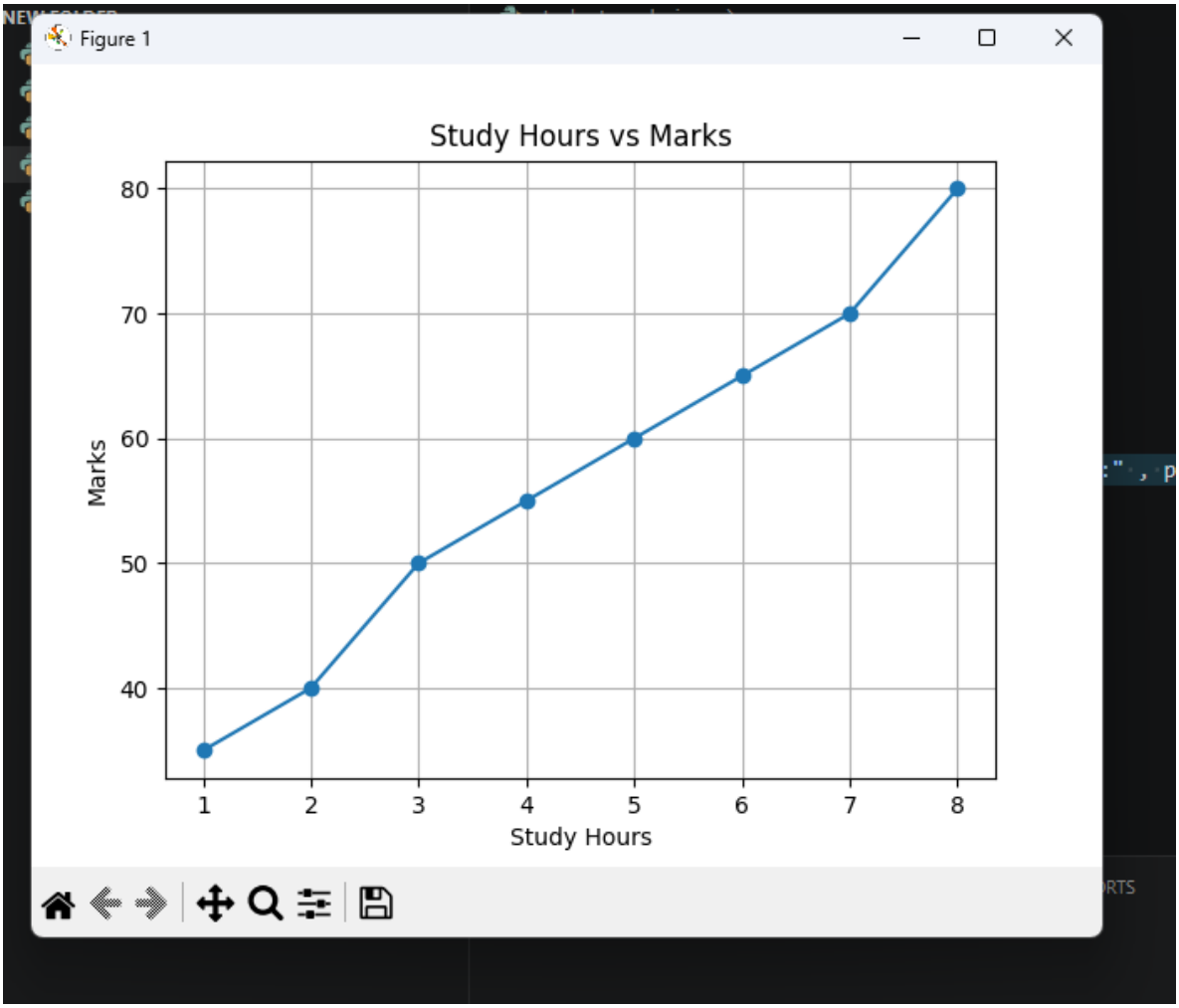
PRACTICAL NO 4
Introduction to Python Programming: Learn the different libraries - NumPy, Pandas, SciPy, Matplotlib, Scikit Learn.

CODE: student_analysis.py

```
import numpy as np
import pandas as pd
from scipy import stats
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

hours = np.array([1,2,3,4,5,6,7,8])

marks = np.array([35,40,50,55,60,65,70,80])

data = pd.DataFrame({
    'Study_Hours': hours,
    'Marks' : marks
})

print("\nStudent Data:")
print(data)

mean_marks = np.mean(marks)
median_marks = np.median(marks)
mode_marks = stats.mode(marks)

print("\nStatistical Analysis")
print("Mean:", mean_marks)
print("Median:", median_marks)
print("Mode:", mode_marks)

plt.plot(hours, marks, marker='o')
plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.grid(True)

plt.show()

x = hours.reshape(-1,1)
y = marks
model = LinearRegression()

model.fit(x, y)

predicted_marks = model.predict([[9]])

print("\nMachihne Learning Prediction")
print("Predicted Marks for 9 Hours Study:" , predicted_marks[0])
```

OUTPUT:

```
Student Data:
   Study_Hours  Marks
0            1     35
1            2     40
2            3     50
3            4     55
4            5     60
5            6     65
6            7     70
7            8     80

Statistical Analysis
Mean: 56.875
Median: 57.5
Mode: ModeResult(mode=np.int64(35), count=np.int64(1))
```



PRACTICAL NO 5
Implement Perceptron algorithm for OR operation

CODE: Percepton operation.py

```
inputs = [(0,0),(0,1),(1,0),(1,1)]
labels = [0,1,1,1]

w1, w2=0.0,0.0
bias = 0.0
learning_rate = 0.1
epochs = 10

def step(x):
   return 1 if x >= 0 else 0

   for _ in range(epochs):
      for (x1,x2), label in zip(inputs, labels):
         linear_output = w1*x1 + w2*x2 + bias
         prediction = step(linear_output)

         error = label - prediction
         w1 += learning_rate * error * x1
         w2 += learning_rate * error * x2
         bias += learning_rate * KeyError

print("Trained weights:" , w1,w2)
print("Bias:", bias)

for x1, x2 in inputs:
   output = step(w1*x1 + w2*x2 +bias )
   print(f"{x1} OR {x2} = {output}")
```

OUTPUT:

PRACTICAL NO 6

Improve the prediction accuracy by estimating the weight values for the training data using stochastic gradient descent. (Perceptron)

CODE:

```
#Perceptron using Stochastic Gradient

import numpy as np
x=np.array([ [0,0],[0,1],[1,0],[1,1]
])

y=np.array([0,0,0,1])

weights=np.zeros(2)
bias=0

lr=0.1 epochs=10

def activation(z):
   return 1 if z>=0 else 0

for epoch in range(epochs):
   print(f"\nEpoch {epoch +1}")

   for i in range(len(x)):
      z=np.dot(x[i],weights)+bias
      y_pred=activation(z)
      error=y[i] - y_pred
      weights = weights + lr * error * x[i]
      bias = bias + lr * error

      print(f"Input: {x[i]}, Predicted: {y_pred}, Error: {error}")

print("\nFinal Weights:", weights)
print("Final Bias:", bias)

print("\nTesting Perceptron")
for i in range(len(x)):
   z = np.dot(x[i], weights) + bias
   y_pred = activation(z)
   print(f"Input: {x[i]} -> Output: {y_pred}")
```

OUTPUT:

```
PS C:\Users\admin\Desktop\AIML> python -u "c:\Users\admin\Desktop\AIML\perceptron using  Stochastic Gradient.py"

Epoch 1
Input: [0 0], Predicted: 1, Error: -1
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 0, Error: 1

Epoch 2
Input: [0 0], Predicted: 1, Error: -1
Input: [0 1], Predicted: 1, Error: -1
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 0, Error: 1

Epoch 3
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 1, Error: -1
Input: [1 0], Predicted: 1, Error: -1
Input: [1 1], Predicted: 0, Error: 1

Epoch 4
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 5
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 6
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 7
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 8
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0
```

```
Epoch 9
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Epoch 10
Input: [0 0], Predicted: 0, Error: 0
Input: [0 1], Predicted: 0, Error: 0
Input: [1 0], Predicted: 0, Error: 0
Input: [1 1], Predicted: 1, Error: 0

Final Weights: [0.2 0.1]
Final Bias: -0.20000000000000004

Testing Perceptron
Input: [0 0] -> Output: 0
Input: [0 1] -> Output: 0
Input: [1 0] -> Output: 0
Input: [1 1] -> Output: 1
```

PRACTICAL NO 7

Implement Adaline algorithm for AND operation

CODE:

```
#Perceptron using Stochastic Gradient

import numpy as np
x=np.array([ [0,0],[0,1],[1,0],[1,1]
])

y=np.array([0,0,0,1])

weights=np.zeros(2)
bias=0

lr=0.1 epochs=10

def activation(z):
    return 1 if z>=0 else 0

for epoch in range(epochs):
    print(f"\nEpoch {epoch +1}")

    for i in range(len(x)):
        z=np.dot(x[i],weights)+bias
        y_pred=activation(z)
        error=y[i] - y_pred
        weights = weights + lr * error * x[i]
        bias = bias + lr * error

        print(f"Input: {x[i]}, Predicted: {y_pred}, Error: {error}")

print("\nFinal Weights:", weights)
print("Final Bias:", bias)

print("\nTesting Perceptron")
for i in range(len(x)):
    z = np.dot(x[i], weights) + bias
    y_pred = activation(z)
    print(f"Input: {x[i]} -> Output: {y_pred}")
```

OUTPUT:

```
PS C:\Users\admin\Desktop\AIML> python -u "c:\Users\admin\Desktop\AIML\Adaline algorithm.py"
Initial Weights:  [0. 0.]
Initial Bias : 0

Epoch 1
Input : [0 0], Target : 0, Output : 0.00
Input : [0 1], Target : 0, Output : 0.00
Input : [1 0], Target : 0, Output : 0.00
Input : [1 1], Target : 1, Output : 0.00
Updated Weights : [0.1 0.1]
Updated Bias : 0.1

Epoch 2
Input : [0 0], Target : 0, Output : 0.10
Input : [0 1], Target : 0, Output : 0.19
Input : [1 0], Target : 0, Output : 0.17
Input : [1 1], Target : 1, Output : 0.22
Updated Weights : [0.16112 0.15922]
Updated Bias : 0.13

Epoch 3
Input : [0 0], Target : 0, Output : 0.13
Input : [0 1], Target : 0, Output : 0.28
Input : [1 0], Target : 0, Output : 0.25
Input : [1 1], Target : 1, Output : 0.33
Updated Weights : [0.20258054 0.19808926]
Updated Bias : 0.13

Epoch 4
Input : [0 0], Target : 0, Output : 0.13
Input : [0 1], Target : 0, Output : 0.32
Input : [1 0], Target : 0, Output : 0.29
Input : [1 1], Target : 1, Output : 0.40
Updated Weights : [0.23371999 0.22650395]
Updated Bias : 0.12

Epoch 5
Input : [0 0], Target : 0, Output : 0.12
Input : [0 1], Target : 0, Output : 0.33
Input : [1 0], Target : 0, Output : 0.31
Input : [1 1], Target : 1, Output : 0.44
Updated Weights : [0.25910386 0.24927607]
Updated Bias : 0.1

Epoch 6
Input : [0 0], Target : 0, Output : 0.10
Input : [0 1], Target : 0, Output : 0.34
Input : [1 0], Target : 0, Output : 0.31
Input : [1 1], Target : 1, Output : 0.47
Updated Weights : [0.28099036 0.26876266]
Updated Bias : 0.08
```

```
Epoch 7
Input : [0 0], Target : 0, Output : 0.08
Input : [0 1], Target : 0, Output : 0.34
Input : [1 0], Target : 0, Output : 0.32
Input : [1 1], Target : 1, Output : 0.49
Updated Weights : [0.3005204  0.28613437]
Updated Bias : 0.06

Epoch 8
Input : [0 0], Target : 0, Output : 0.06
Input : [0 1], Target : 0, Output : 0.34
Input : [1 0], Target : 0, Output : 0.32
Input : [1 1], Target : 1, Output : 0.51
Updated Weights : [0.31829198 0.30198762]
Updated Bias : 0.03

Epoch 9
Input : [0 0], Target : 0, Output : 0.03
Input : [0 1], Target : 0, Output : 0.33
Input : [1 0], Target : 0, Output : 0.32
Input : [1 1], Target : 1, Output : 0.52
Updated Weights : [0.33463769 0.31664003]
Updated Bias : 0.01

Epoch 10
Input : [0 0], Target : 0, Output : 0.01
Input : [0 1], Target : 0, Output : 0.33
Input : [1 0], Target : 0, Output : 0.31
Input : [1 1], Target : 1, Output : 0.53
Updated Weights : [0.34975884 0.3302731 ]
Updated Bias : -0.01

Testing Adaline for AND Operation
Input : [0 0] -> Output : 0
Input : [0 1] -> Output : 0
Input : [1 0] -> Output : 0
Input : [1 1] -> Output : 1
```

Machine Learning Section

PRACTICAL NO 1

Implementation of Features Extraction and Selection, Normalization, Transformation, Principal Components Analysis.

CODE:

```
import pandas as pd import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, PowerTransformer, LabelEncoder from sklearn.feature_selection import SelectKBest, f_classif, VarianceThreshold
from sklearn.decomposition import PCA



# Load the healthcare dataset file_path = "health_dataset.csv" df = pd.read_csv(file_path)

# Display first few rows
display(df.head())



# Identify non-numeric columns
categorical_columns = df.select_dtypes(include=['object']).columns print("Categorical Columns:", categorical_columns)

# Convert categorical columns using Label Encoding
label_encoders = {}
for col in categorical_columns: le = LabelEncoder()
df[col] = le.fit_transform(df[col]) label_encoders[col] = le

# Ensure all columns are numeric
df = df.apply(pd.to_numeric, errors='coerce') # Converts non-numeric values to NaN df.fillna(0, inplace=True) # Replace NaNs with 0

# Separate features and target
target_column = 'Disease Risk Score' if target_column not in df.columns:
raise ValueError("Target column not found in dataset")



X = df.drop(columns=[target_column]) y = df[target_column]

# Check for missing values in target and encode if necessary
if y.isnull().sum() > 0:
y.fillna(y.mode()[0], inplace=True) # Replace NaN with most frequent value
```

```
if y.dtype == 'object': le = LabelEncoder()
y = le.fit_transform(y)


# Remove constant features
var_thresh = VarianceThreshold(threshold=0) X = var_thresh.fit_transform(X)

# Feature Selection using ANOVA F-score
selector = SelectKBest(score_func=f_classif, k=min(8, X.shape[1])) # Ensure k does not exceed feature count
X_selected = selector.fit_transform(X, y) print(X_selected)
selected_features = [col for col, keep in zip(df.drop(columns=[target_column])
.columns,
selector.get_support()) if keep] print("Selected Features:", selected_features)

# Normalization using Min-Max Scaling
scaler = MinMaxScaler()
X_normalized = scaler.fit_transform(X_selected)


# Transformation using Power Transform (Box-Cox or Yeo-Johnson)
power_transformer = PowerTransformer(method='yeo-johnson') # Use 'box-cox' if no negative values
X_transformed = power_transformer.fit_transform(X_normalized)


# Dimensionality Reduction using PCA
pca_components = min(5, X_transformed.shape[1]) # Ensure PCA components do not exceed feature count
pca = PCA(n_components=pca_components) X_pca = pca.fit_transform(X_transformed)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)


# Convert processed data back to DataFrame
processed_df = pd.DataFrame(X_pca, columns=[f'PC{i+1}'
for i in range(X_pca.shape[1])])
processed_df['Disease Risk Score'] = y.reset_index(drop=True)


# Save processed dataset
processed_df.to_csv("processed_health_dataset.csv", index=False) print("Processed dataset saved successfully.")
```

OUTPUT:

	ID	Age	Height (cm)	Weight (kg)	Blood Pressure (BP)	Cholesterol Level	Diabetes	Physical Activity (hours/week)	Smoking Habit	Disease Risk Score
0	1	56	159	77	122	High	1	1.00	0	59.00
1	2	69	185	77	157	Normal	0	0.53	1	48.00
2	3	46	163	93	122	Low	1	9.59	0	42.05
3	4	32	180	79	103	Normal	1	8.47	0	78.47
4	5	60	197	111	110	Normal	1	3.55	1	63.94

```
Categorical Columns: Index(['Cholesterol Level'], dtype='object')
[[ 56.   159.    77.   122.     0.     1.     1.     0.  ]
 [ 69.   185.    77.   157.     2.     0.     0.53   1.  ]
 [ 46.   163.    93.   122.     1.     1.     9.59   0.  ]
 [ 32.   180.    79.   103.     2.     1.     8.47   0.  ]
 [ 60.   197.   111.   110.     2.     1.     3.55   1.  ]
 [ 25.   164.   111.   137.     2.     0.     9.57   1.  ]
 [ 78.   157.    50.   109.     2.     1.     6.77   0.  ]
 [ 38.   163.    76.    97.     1.     1.     4.83   1.  ]
 [ 56.   172.   111.    96.     2.     1.     4.93   0.  ]
 [ 75.   189.    52.   156.     2.     1.     0.83   0.  ]
 [ 36.   170.   119.   106.     0.     0.     0.92   0.  ]
 [ 40.   165.    76.   122.     2.     1.     6.02   1.  ]
 [ 28.   194.    58.   137.     0.     0.     5.54   1.  ]
 [ 28.   167.   111.   165.     1.     0.     2.13   1.  ]
 [ 41.   196.    86.   148.     1.     0.     9.46   1.  ]
 [ 70.   173.   100.   175.     1.     0.     7.81   1.  ]
 [ 53.   175.    93.   111.     1.     0.     1.13   0.  ]
 [ 57.   174.    73.   119.     0.     0.     9.31   0.  ]
 [ 41.   194.   108.   127.     2.     1.     9.74   0.  ]
 [ 20.   190.    81.   140.     1.     0.     9.96   0.  ]
 [ 39.   178.   101.   143.     2.     0.     0.56   1.  ]
 [ 70.   164.   111.    97.     0.     0.     7.37   0.  ]
 [ 19.   194.   107.   116.     0.     1.     5.46   1.  ]
 [ 41.   150.   101.   116.     0.     1.     7.06   0.  ]
 [ 61.   174.    61.   110.     1.     0.     9.69   1.  ]
 [ 47.   156.    88.   119.     2.     1.     6.88   1.  ]
 [ 55.   158.    51.   117.     2.     0.     8.37   0.  ]
 [ 19.   173.    52.   153.     2.     0.     8.67   0.  ]
 [ 77.   150.   105.   158.     0.     1.     8.38   0.  ]
 [ 38.   193.   108.   150.     1.     0.     4.26   0.  ]
 [ 50.   157.    51.   137.     0.     1.     2.23   1.  ]
 [ 29.   173.    51.   108.     0.     1.     3.97   0.  ]
 [ 75.   160.   103.    93.     2.     0.     8.92   0.  ]
 [ 39.   166.    50.   124.     0.     0.     1.47   1.  ]
 [ 78.   157.    68.   153.     2.     1.     5.13   0.  ]
 [ 61.   184.    51.   138.     1.     0.     2.33   0.  ]
```

```
[ 42.   184.   102.   106.     2.     0.     5.81   0.  ]
[ 66.   182.    93.   133.     1.     0.     8.63   0.  ]
[ 44.   154.    81.   119.     2.     1.     8.8    0.  ]
[ 76.   191.   119.   135.     0.     1.     2.37   1.  ]
[ 59.   188.    81.    95.     0.     0.     9.08   0.  ]
[ 45.   190.   117.   126.     1.     1.     5.92   0.  ]
[ 77.   177.   104.   113.     1.     1.     3.5    0.  ]
[ 33.   156.   105.   135.     1.     1.     7.08   0.  ]
[ 32.   158.    66.   142.     2.     1.     4.82   0.  ]
[ 79.   157.    87.   149.     2.     1.     3.78   1.  ]
[ 79.   161.    73.   152.     1.     0.     7.05   0.  ]
[ 64.   183.   118.   174.     0.     0.     2.49   1.  ]
[ 79.   182.   119.   121.     2.     0.     3.3    1.  ]
[ 68.   197.    60.   176.     2.     1.     4.34   0.  ]
[ 61.   172.    65.   122.     2.     1.     2.54   0.  ]
[ 72.   173.   108.   156.     1.     0.     4.05   0.  ]
[ 69.   186.   119.   107.     1.     1.     5.71   0.  ]
[ 74.   184.    52.   114.     1.     0.     7.41   1.  ]
[ 20.   193.    69.   143.     1.     0.     7.67   0.  ]
[ 54.   189.   108.   147.     1.     0.     8.23   0.  ]
[ 68.   171.    85.   156.     0.     1.     7.44   0.  ]
[ 24.   176.    68.   135.     0.     1.     6.81   0.  ]
[ 38.   184.   116.   113.     1.     0.     2.38   0.  ]
[ 26.   150.    68.   121.     2.     0.     4.     0.  ]
[ 56.   184.    69.   136.     1.     0.     4.78   1.  ]
[ 35.   186.   101.   175.     0.     0.     0.83   1.  ]
[ 21.   196.    82.   112.     0.     1.     5.28   0.  ]
[ 42.   163.    89.   155.     2.     1.     4.36   0.  ]
[ 77.   152.    88.   116.     0.     0.     8.02   0.  ]
[ 31.   150.    50.    91.     1.     0.     9.78   1.  ]
[ 67.   154.    60.   179.     2.     1.     5.56   0.  ]
[ 75.   175.   106.   106.     1.     1.     3.23   1.  ]
[ 26.   163.    99.   122.     1.     0.     0.43   0.  ]
[ 43.   188.    72.    98.     2.     0.     9.25   0.  ]
[ 70.   176.    80.   132.     1.     0.     9.19   0.  ]
[ 19.   158.    91.   137.     0.     1.     2.53   0.  ]
[ 37.   164.    56.   128.     1.     1.     6.95   1.  ]
[ 45.   164.    65.   131.     2.     1.     0.75   1.  ]
[ 64.   175.   109.   115.     2.     0.     1.66   0.  ]
[ 77.   191.    51.   139.     2.     0.     2.17   0.  ]
[ 24.   162.    50.   114.     1.     1.     2.94   1.  ]
[ 61.   181.    97.   113.     2.     1.     9.96   1.  ]
[ 78.   188.    61.   102.     2.     0.     6.97   1.  ]
[ 25.   198.   118.   149.     1.     1.     3.84   1.  ]
[ 64.   181.    86.    96.     0.     1.     7.37   0.  ]
[ 52.   153.    81.   146.     1.     0.     9.15   1.  ]
[ 31.   179.    58.   125.     2.     1.     9.59   0.  ]
[ 34.   186.    68.   134.     2.     0.     0.58   1.  ]
[ 53.   172.    97.   109.     1.     0.     3.95   1.  ]
```

```
 [ 67.   188.    52.   154.     0.     0.     1.07   1.  ]
 [ 57.   194.    69.    97.     2.     0.     3.36   1.  ]
 [ 21.   164.    73.   105.     2.     0.     1.7    1.  ]
 [ 19.   192.   103.   103.     2.     0.     6.47   0.  ]
 [ 79.   178.    82.   165.     1.     0.     3.88   0.  ]
 [ 23.   185.    73.   176.     2.     1.     2.29   1.  ]
 [ 71.   162.    85.   104.     0.     0.     2.66   1.  ]
 [ 59.   181.    87.   155.     2.     0.     3.6    0.  ]
 [ 21.   156.    74.   121.     2.     1.     2.6    1.  ]
 [ 71.   171.    67.   176.     0.     1.     4.53   1.  ]
 [ 46.   177.   115.   152.     2.     1.     0.32   0.  ]
 [ 35.   151.   103.   175.     1.     0.     2.8    1.  ]
 [ 43.   191.    84.   140.     1.     1.     4.11   1.  ]
 [ 61.   194.   110.   114.     1.     0.     6.03   0.  ]
 [ 51.   155.    90.   147.     0.     0.     2.71   0.  ]]
Selected Features: ['Age', 'Height (cm)', 'Weight (kg)', 'Blood Pressure (BP)', 'Cholesterol Level', 'Diabetes', 'Physical Activity (hours/week)', 'Smo
king Habit']
Explained Variance Ratio:
[0.17300838 0.15542009 0.13356947 0.12778434 0.12436891]
Processed dataset saved successfully.
```

PRACTICAL NO 2

Implementation of Logistic regression

CODE:

```
import numpy as np
from sklearn import datasets iris = datasets.load_iris() print(type(iris))
print(list(iris.keys()))
X = iris["data"][:,3:] # petal width
y = (iris["target"] == 2).astype(np.int64) # 1 if Iris-Virginica, else 0
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(solver="lbfgs", random_state=42) log_reg.fit(X,y)
import matplotlib.pyplot as plt
X_new = np.linspace(0,3,1000).reshape(-1,1) y_proba = log_reg.predict_proba(X_new)
plt.plot(X_new, y_proba[:,1],"g-")
plt.plot(X_new, y_proba[:,0], "b--") plt.xlabel('Petal width (cm)') plt.ylabel('Probability')
plt.legend(['Prob(Iris-Virginica)','Prob(Not Iris-Virginica)']) print(log_reg.predict(X))
print(log_reg.coef_) print(log_reg.intercept_)
```

2. Implementation of Logistic regression (two features)

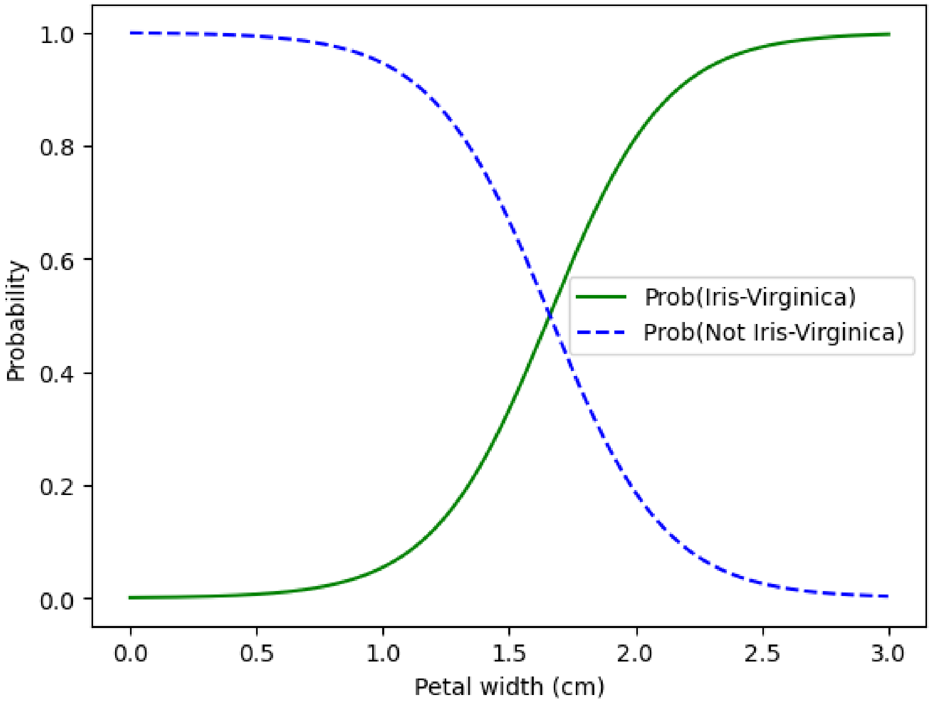
#Problem Statement 2: Logistic Regression for predicting class using two features: Petal length and width.

CODE:

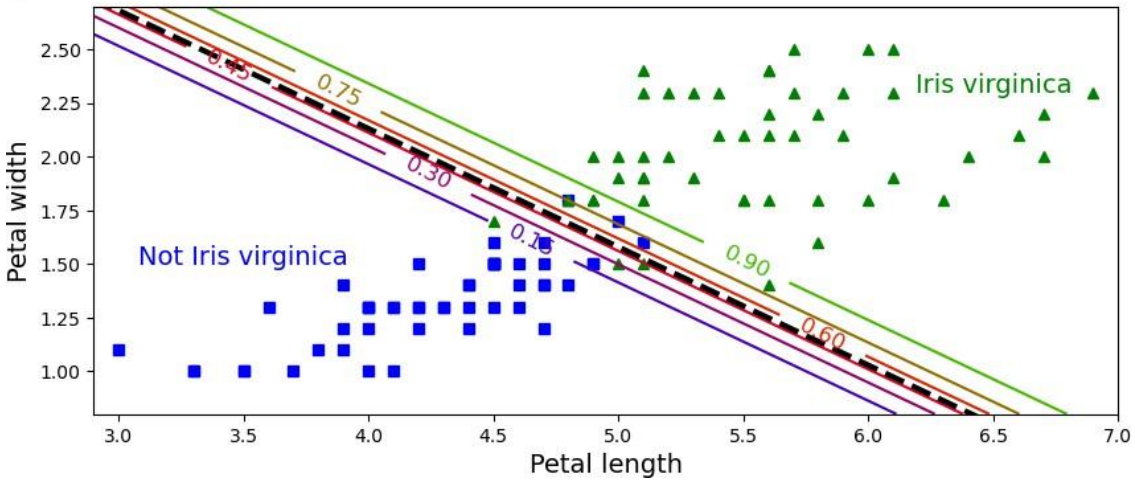
```
from sklearn.linear_model import LogisticRegression X = iris["data"][:, (2, 3)] # petal
length, petal width y = (iris["target"] == 2).astype(np.int64)
log_reg2 = LogisticRegression(solver="lbfgs", C=10**10, random_state=42)
log_reg2.fit(X, y)
x0, x1 = np.meshgrid(
np.linspace(2.9, 7, 500).reshape(-1, 1),
np.linspace(0.8, 2.7, 200).reshape(-1, 1),
)
X_new = np.c_[x0.ravel(), x1.ravel()] print(X_new.shape)
y_proba = log_reg2.predict_proba(X_new) plt.figure(figsize=(10, 4))
plt.plot(X[y==0, 0], X[y==0, 1], "bs")
plt.plot(X[y==1, 0], X[y==1, 1], "g^") zz = y_proba[:, 1].reshape(x0.shape)
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg) left_right = np.array([2.9, 7])
boundary = -(log_reg2.coef_[0][0] * left_right + log_reg2.intercept_[0]) /
log_reg2.coef_[0][1] plt.clabel(contour, inline=1, fontsize=12)
plt.plot(left_right, boundary, "k--", linewidth=3)
plt.text(3.5, 1.5, "Not Iris virginica", fontsize=14, color="b", ha="center") plt.text(6.5, 2.3,
"Iris virginica", fontsize=14, color="g", ha="center") plt.xlabel("Petal length", fontsize=14)
plt.ylabel("Petal width", fontsize=14) plt.axis([2.9, 7, 0.8, 2.7])
```

OUTPUT:

```
<class 'sklearn.utils._bunch.Bunch'>
['data', 'target', 'frame', 'target_names', 'DESCR', 'feature_names', 'filename', 'data_module']
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 0 1 1 1 1 1 1 1 1 1 0 1 1 1 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1]
[[4.33251974]]
[-7.19342106]
```



```
(100000, 2)
(np.float64(2.9), np.float64(7.0), np.float64(0.8), np.float64(2.7))
```



PRACTICAL NO 3

Implementation of Classifying data using Support Vector Machine (SVM)- Linear and Non-Linear SVM Classification

1. Linear SVM

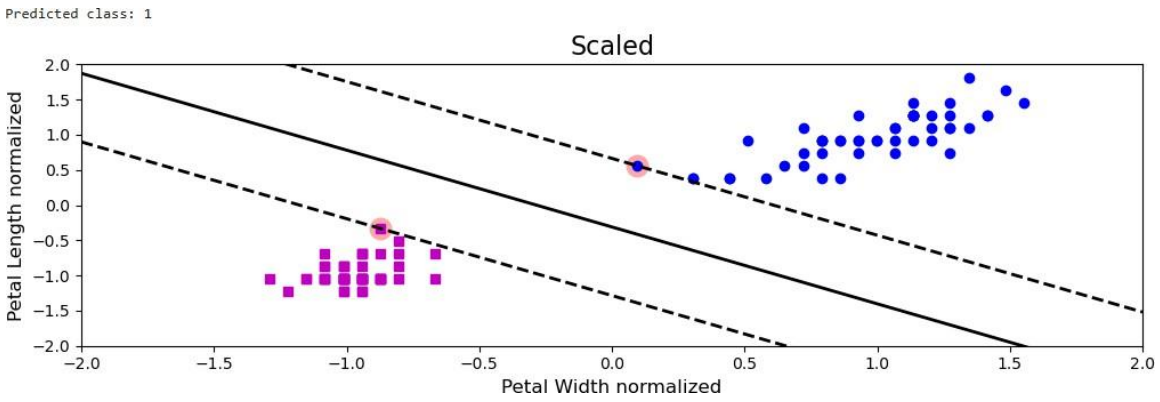
```
from sklearn.svm import SVC from sklearn import datasets import numpy as np
# Load Iris dataset
iris = datasets.load_iris()
X = iris["data"][:, (2, 3)] # Select petal length and petal width
y = iris["target"]
# Select only Setosa and Versicolor classes
setosa_or_versicolor = (y == 0) | (y == 1) X = X[setosa_or_versicolor]
y = y[setosa_or_versicolor]


# SVM Classifier model with a large but finite C value
svm_clf = SVC(kernel="linear", C=1e10) # Large C approximates a hard margin
svm_clf.fit(X, y)


# Make a prediction
prediction = svm_clf.predict([[2.4, 3.1]]) print("Predicted class:", prediction[0])
import matplotlib
import matplotlib.pyplot as plt
def plot_svc_decision_boundary(svm_clf, xmin, xmax): w = svm_clf.coef_[0]
b = svm_clf.intercept_[0]
# At the decision boundary, w0*x0 + w1*x1 + b = 0 # => x1 = -w0/w1 * x0 - b/w1
x0 = np.linspace(xmin, xmax, 200)

decision_boundary = -w[0]/w[1] * x0 - b/w[1] margin = 1/w[1]
gutter_up = decision_boundary + margin gutter_down = decision_boundary - margin svs =
svm_clf.support_vectors_
plt.scatter(svs[:, 0], svs[:, 1], s=180, facecolors='#FFAAAA') plt.plot(x0, decision_boundary, "k-",
linewidth=2) plt.plot(x0, gutter_up, "k--", linewidth=2)
plt.plot(x0, gutter_down, "k--", linewidth=2)
#plot the decision boundaries import numpy as np plt.figure(figsize=(12,3.2))
from sklearn.preprocessing import StandardScaler scaler = StandardScaler()
X_scaled = scaler.fit_transform(X) svm_clf.fit(X_scaled, y)
plt.plot(X_scaled[:, 0][y==1], X_scaled[:, 1][y==1], "bo")
plt.plot(X_scaled[:, 0][y==0], X_scaled[:, 1][y==0], "ms") plot_svc_decision_boundary(svm_clf, -2,
2)
plt.xlabel("Petal Width normalized", fontsize=12) plt.ylabel("Petal Length normalized", fontsize=12)
plt.title("Scaled", fontsize=16)
plt.axis([-2, 2, -2, 2])
```

2. Non-Linear SVM Code:

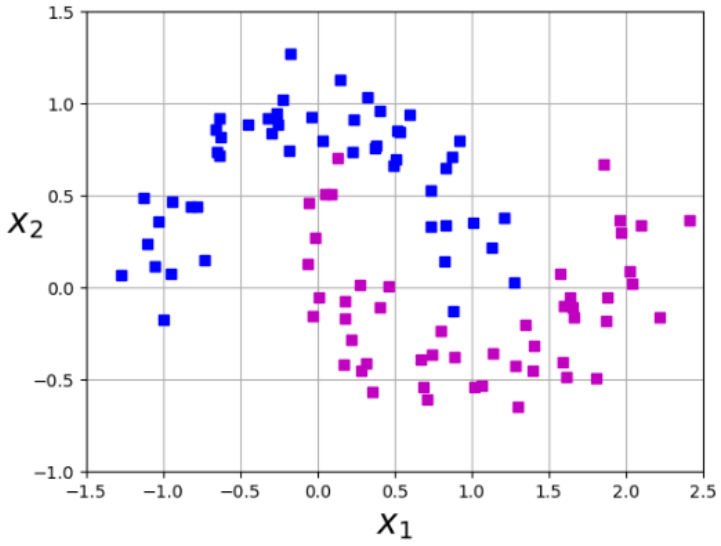
```
from sklearn.datasets import make_moons from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures from sklearn.preprocessing import
StandardScaler
from sklearn.svm import SVC import numpy as np
```

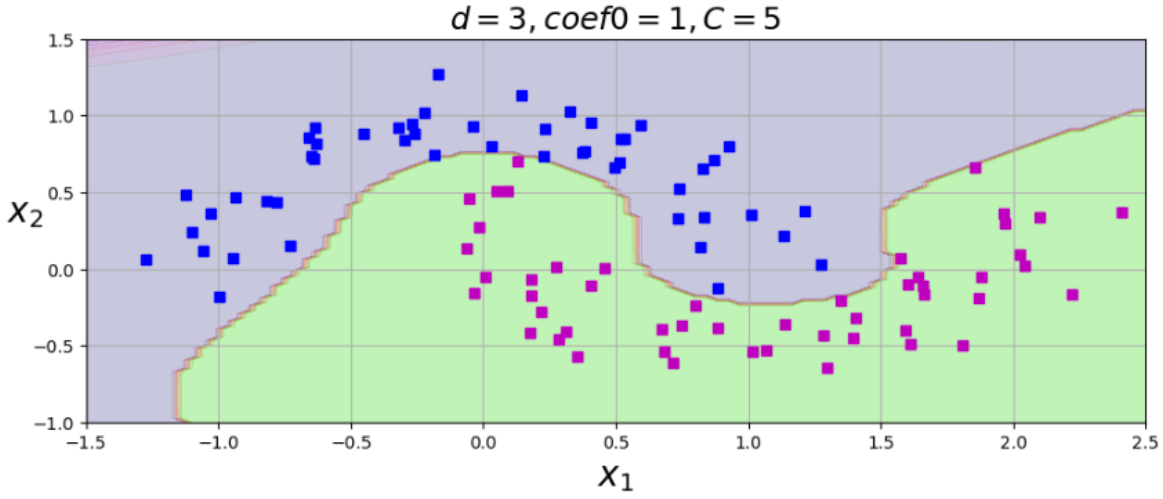
```
%matplotlib inline import matplotlib
import matplotlib.pyplot as plt
X, y = make_moons(n_samples=100, noise=0.15, random_state=42) print(X.shape)
print(y.shape)
#define a function to plot the dataset def plot_dataset(X, y, axes):
plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
plt.plot(X[:, 0][y==1], X[:, 1][y==1], "ms")
plt.axis(axes)
plt.grid(True, which='both') plt.xlabel(r"$x_1$", fontsize=20)
plt.ylabel(r"$x_2$", fontsize=20, rotation=0) #Let's have a look at the data we have generated
plot_dataset(X, y, [-1.5, 2.5, -1, 1.5])
#C controls the width of the street #Degree of data
#create a pipeline to create features, scale data and fit the model
polynomial_svm_clf = Pipeline((
("poly_features", PolynomialFeatures(degree=3)), ("scalar", StandardScaler()),
("svm_clf", SVC(kernel="poly", degree=10, coef0=1, C=5))
))
#call the pipeline
polynomial_svm_clf.fit(X,y)
#define a function plot the decision boundaries
def plot_predictions(clf, axes):
#create data in continous linear space x0s = np.linspace(axes[0], axes[1], 100) x1s =
np.linspace(axes[2], axes[3], 100) x0, x1 = np.meshgrid(x0s, x1s)
X = np.c_[x0.ravel(), x1.ravel()]
y_pred = clf.predict(X).reshape(x0.shape)
y_decision = clf.decision_function(X).reshape(x0.shape) plt.contourf(x0, x1, y_pred,
cmap=plt.cm.brg, alpha=0.2)
plt.contourf(x0, x1, y_decision, cmap=plt.cm.brg, alpha=0.1)
#plot the decision boundaries
plt.figure(figsize=(11, 4))
#plot the dataset
plot_dataset(X, y, [-1.5, 2.5, -1, 1.5])
#plot the decision boundaries
plot_predictions(polynomial_svm_clf, [-1.5, 2.5, -1, 1.5]) plt.title(r"$d=3, coef0=1, C=5$",
fontsize=18)
```

OUTPUT:



```
(100, 2)
(100,)
```


1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5
x_2
x_1


$d = 3, coef0 = 1, C = 5$
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
2.5
x_2
x_1

PRACTICAL NO 4

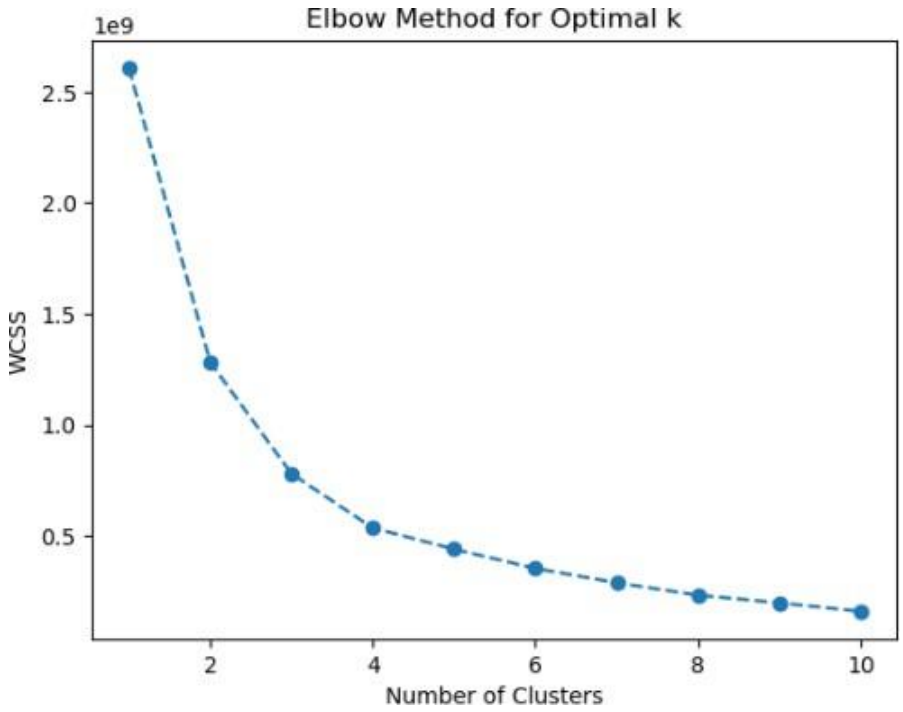
Implementation of Bagging Algorithm: Random Forest

CODE:

```
import pandas as pd import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Load the dataset
df = pd.read_csv("clustering.csv")
# Display first few rows of the dataset
print(df.head())
# Drop missing values
df_cleaned = df.dropna()
# Selecting numerical columns for clustering
numerical_cols = df_cleaned.select_dtypes(include=[np.number]).columns print("Numerical columns used for clustering:", numerical_cols.tolist())
# Feature selection for clustering (Modify as needed)
X = df_cleaned[numerical_cols]
# Apply the Elbow Method
wcss = [] # Within-cluster sum of squares
for i in range(1, 11): # Trying different cluster numbers from 1 to 10
kmeans = KMeans(n_clusters=i, random_state=42, n_init=10) kmeans.fit(X)
wcss.append(kmeans.inertia_)
# Plot the Elbow Method
plt.plot(range(1, 11), wcss, marker='o', linestyle='--') plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal k') plt.show()
# Choose optimal k (Modify based on the elbow plot observation)

k_optimal = 3 # Example choice, change based on your dataset # Apply K-Means with the optimal number of clusters
kmeans = KMeans(n_clusters=k_optimal, random_state=42, n_init=10) df_cleaned['Cluster'] = kmeans.fit_predict(X)
# Display clustered data
print(df_cleaned.head())
# Plot the clusters (for 2D visualization, choose two relevant features)
plt.scatter(df_cleaned[numerical_cols[0]], df_cleaned[numerical_cols[1]], c=df_cleaned['Cluster'], cmap='viridis')
plt.xlabel(numerical_cols[0])  plt.ylabel(numerical_cols[1])
plt.title(f'K-Means Clustering (k={k_optimal})') plt.colorbar(label='Cluster')
plt.show()
```

OUTPUT:

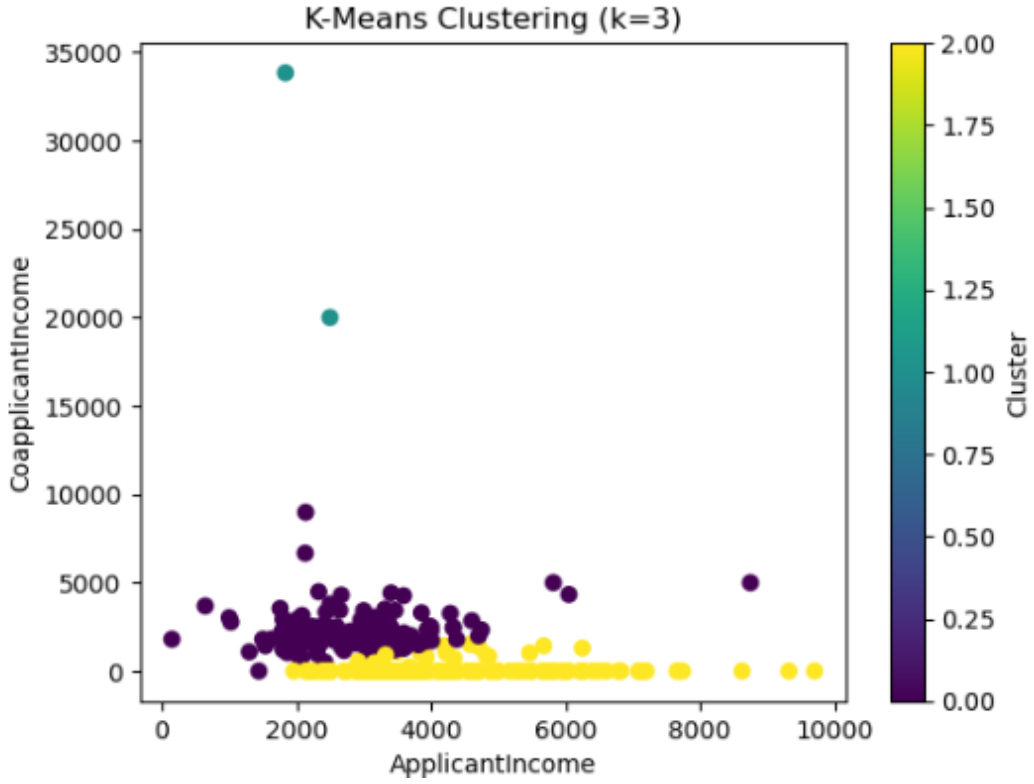
```
    Loan_ID Gender Married Dependents     Education Self_Employed  \
0  LP001003   Male     Yes          1      Graduate            No
1  LP001005   Male     Yes          0      Graduate           Yes
2  LP001006   Male     Yes          0  Not Graduate            No
3  LP001008   Male      No          0      Graduate            No
4  LP001013   Male     Yes          0  Not Graduate            No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0             4583             1508.0       128.0             360.0
1             3000                0.0        66.0             360.0
2             2583             2358.0       120.0             360.0
3             6000                0.0       141.0             360.0
4             2333             1516.0        95.0             360.0

   Credit_History Property_Area Loan_Status
0             1.0         Rural           N
1             1.0         Urban           Y
2             1.0         Urban           Y
3             1.0         Urban           Y
4             1.0         Urban           Y
Numerical columns used for clustering: ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History']
```



	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001003	Male	Yes	1	Graduate	No	
1	LP001005	Male	Yes	0	Graduate	Yes	
2	LP001006	Male	Yes	0	Not Graduate	No	
3	LP001008	Male	No	0	Graduate	No	
4	LP001013	Male	Yes	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	4583	1508.0	128.0	360.0	
1	3000	0.0	66.0	360.0	
2	2583	2358.0	120.0	360.0	
3	6000	0.0	141.0	360.0	
4	2333	1516.0	95.0	360.0	

	Credit_History	Property_Area	Loan_Status	Cluster
0	1.0	Rural	N	2
1	1.0	Urban	Y	2
2	1.0	Urban	Y	0
3	1.0	Urban	Y	2
4	1.0	Urban	Y	0



PRACTICAL NO 5
Implementation of Bagging Algorithm: Random Forest

CODE:

```
import numpy as np import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score,confusion_matrix from sklearn.decomposition import PCA
# Load dataset
iris = load_iris()
X = iris.data y = iris.target
# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Initialize and train the Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42) rf_classifier.fit(X_train,
y_train)

# Make predictions
y_pred = rf_classifier.predict(X_test)


# Evaluate the model
cm=confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy of Random Forest Classifier: {accuracy * 100:.2f}%') print("Confusion Matrix:\n", cm)
```

OUTPUT:

```
Accuracy of Random Forest Classifier: 100.00%
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

PRACTICAL NO 6

Implementation of Boosting Algorithms: AdaBoost, Stochastic Gradient Boosting, Voting Ensemble

CODE:

```
import numpy as np import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier,
GradientBoostingClassifier, VotingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression from sklearn.metrics import accuracy_score
from sklearn.decomposition import PCA
# Load dataset
iris = load_iris()
X = iris.data y = iris.target

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42) rf_classifier.fit(X_train,
y_train)
y_pred_rf = rf_classifier.predict(X_test)
accuracy_rf = accuracy_score(y_test, y_pred_rf)
print(f'Accuracy of Random Forest Classifier: {accuracy_rf * 100:.2f}%')


# AdaBoost Classifier
adaboost = AdaBoostClassifier(base_estimator=DecisionTreeClassifier(max_depth=1),
n_estimators=50, random_state=42)

adaboost.fit(X_train, y_train)
y_pred_adaboost = adaboost.predict(X_test)
accuracy_adaboost = accuracy_score(y_test, y_pred_adaboost)
print(f'Accuracy of AdaBoost Classifier: {accuracy_adaboost * 100:.2f}%')


# Gradient Boosting Classifier
gb_classifier = GradientBoostingClassifier(n_estimators=100,
```

```
learning_rate=0.1, random_state=42, subsample=0.8) gb_classifier.fit(X_train, y_train)
y_pred_gb = gb_classifier.predict(X_test)
accuracy_gb = accuracy_score(y_test, y_pred_gb)
print(f'Accuracy of Gradient Boosting Classifier: {accuracy_gb * 100:.2f}%')


# Voting Classifier (Ensemble of Logistic Regression, Decision Tree, and Random Forest)
voting_classifier = VotingClassifier(estimators=[ ('lr', LogisticRegression()),
('dt', DecisionTreeClassifier()),
('rf', RandomForestClassifier(n_estimators=100))
], voting='hard')
voting_classifier.fit(X_train, y_train)
y_pred_voting = voting_classifier.predict(X_test)
accuracy_voting = accuracy_score(y_test, y_pred_voting)
print(f'Accuracy of Voting Classifier: {accuracy_voting * 100:.2f}%')


# Reduce dimensions for visualization pca = PCA(n_components=2) X_reduced =
pca.fit_transform(X)

# Scatter plot of the dataset
plt.figure(figsize=(8, 6))
plt.scatter(X_reduced[:, 0], X_reduced[:, 1], c=y, cmap='viridis', edgecolor='k', alpha=0.7)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('Iris Dataset Visualization with PCA') plt.colorbar(label='Class Labels')
plt.show()
```

OUTPUT:

```
Accuracy of Random Forest Classifier: 100.00%
Accuracy of AdaBoost Classifier: 93.33%
Accuracy of Gradient Boosting Classifier: 100.00%
Accuracy of Voting Classifier: 100.00%
```